Technical report - Apertus Mundus

- Track: Track 2b - Own Project - Team: Apertus Mundus - Demo video: not yet recorded - Source revision: the Git commit containing this report

1. Summary

Apertus Mundus supports a GM running a persistent roleplaying world. It separates candidate summaries, selected-branch review, expanded narrative and transactional commit. Teams share authoritative world facts while preserving scoped memories.

2. Architecture

`make run` builds four Docker services: the Python/FastAPI workbench, BGE-M3 embedding, MXBAI Rerank Base v2, and Kev-4B decision inference. The workbench connects to the judges' Apertus endpoint through `LLM_NAME`, `LLM_BASE_URL`, and `LLM_API_KEY`. Only the workbench's host loopback port is published. SQLite, settings and saves live in a persistent Docker volume. The original CC0 demo is initialized once.

Target architecture

Self-managed on-premise deployment is provided by `make run-local`: an additional official Apertus v1.5 8B GPU service replaces the hosted Apertus connection. The auxiliary services run locally in both modes. The API demonstration mode relies on the supplied Apertus endpoint; its location is not inferred from its model name. We do not claim air-gapped certification or a verified sovereign-cloud deployment.

Build time requires package registries and public model downloads. Auxiliary weights are baked into images and loaded offline at runtime. Optional local Apertus weights are prepared separately after accepting the model's access conditions. Browser fonts and icons may use optional online resources. See deployment guide.

3. Use of Apertus and supporting models

Apertus performs summary generation, narrative expansion, Wiki synthesis, and decision fallback. The hosted model is selected by `LLM_NAME`; local mode defaults to `swiss-ai/Apertus-v1.5-8B`. Prompts and schema validation are in the source.

`BAAI/bge-m3` supplies 1024-dimensional dense embeddings. Permissions and world state are filtered before retrieval. `mixedbread-ai/mxbai-rerank-base-v2` reranks up to 80 candidate evidence fragments. `jaredpalmer/kev-4b`, on `Qwen/Qwen3.5-4B-Base`, serves typed System One decisions and probabilities, falling back to Apertus on uncertainty. Kev is a separate open model; it is not the hosted Jev model. Its English evaluation does not establish quality on Chinese roleplaying text.

All auxiliary model revisions are recorded in `helpers/models.json`; upstream Kev code is fixed to `1f04cfa28e6a708485b28a9585010b9e436b776e`. BGE-M3 uses MIT and the selected MXBAI/Kev/base models use Apache-2.0. Downloaded model cards and licences are retained.

4. Data

The distributable includes only an original fictional lighthouse scenario, map and maintenance notes, released under CC0. It excludes the developer's real play sessions, private campaign documents, production database and API credentials. Total `data/` size is below 100 MB. Model weights are downloaded at build time rather than committed.

5. Evaluation

The deployment smoke harness uses an isolated temporary world. Offline mode replaces provider IO and checks the application chain; live mode additionally verifies real embedding, reranking, Kev decisions and Apertus generation. It tests failed-generation isolation, commit idempotence, exported-save loading and fresh-process

database recovery.

The restructured source passes the isolated offline smoke test, including commit and export recovery. Docker Compose configuration and root Makefile forwarding also pass. Record final container-build, helper-inference and live end-to-end results here before submission. The prior hosted listwise reranker evaluation does not measure the local Base v2 model. Mock outputs are not evidence of model quality or real latency.

6. Limitations

CPU Kev-4B can require substantial RAM and inference time. Docker RAM capacity and GPU availability vary by evaluator. Exact minimum hardware, model quality, memory peaks and live end-to-end latency require measurement on the target deployment. The current development Docker engine exposes about 7.36 GiB RAM and an 8 GiB GPU; this is insufficient evidence for a successful full-size concurrent helper deployment. The report must identify any validation still pending rather than treating configuration or a successful image build as a completed real-model run.

7. Reproducibility

Use Docker Engine 28+, Compose 2.24+ and GNU Make. Run `make run` with the supplied Apertus configuration, then `make smoke` and `make smoke-live`. Supporting model weights total 12,716,349,435 bytes based on the fixed Hub metadata (2026-10-16), excluding tokenizers and runtime dependencies. `python helpers/model_inventory.py` repeats the inventory. `make package` exports the public source tree and a SHA-256 file manifest. Record final image digests, package versions and tested hardware alongside results.

8. Next steps

Evaluate local-model Chinese decisions and retrieval quality, measure concurrency and resource needs, and validate a fully disconnected deployment separately.

References

- Track 2b requirements - Apertus v1.5 - BGE-M3 - MXBAI Base v2 - Kev model and source